

4 Objects and classes

4.1 Objects

A program is composed of generic objects, with certain standard properties, and certain standard operations the objects can perform. In object oriented programming (OOP), you only care about what the objects expose. You won't be able to determine how someone else's object functions. In fact, the public method (interface) is more important.

In traditional procedural programming, you search for verbs (actions) in the problem definition. In procedural programming, verbs directly suggest procedures and then, lastly, you think of data variables to go with those procedures. In OOP, you put data structures first, and then look at the algorithms that operate on the data. The secret to effective OOP is that each object carries out a small set of related tasks. If an object needs a task done but that task isn't the job of that object then that object asks another object to do the task.

Again, since the first object can't do the task, it asks the second object to carry out the task. In OOP, one object must never directly manipulate the internal data of another object. Rather, all communication is via "messages". A message is another name for a method call.

4.2 Classes

The term class is the blueprint or recipe from which the object is actually made, or "instantiated."

```
MyClass boop;  
boop = new MyClass();
```

We are now familiar with this, the first "MyClass boop;" makes a reference called "boop." At this point, the reference called "boop" does not actually point to any existing object. Soon, it will point to an object of type MyClass, but now the object doesn't exist.

```
MyClass boop = new MyClass();
```

When this statement executes, the new keyword executes the default Constructor for MyClass, which actually creates an object in memory and assigns that reference to boop. The handle to that just-created object is given to the MyClass reference boop. Now boop points to the new MyClass object.

State behavior identity